

DATABASE DESIGN DOCUMENTATION

WitleShop Online Retail System

Ntwanano Phiona Shabane

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Method	3
2. Summary of Business Requirements	3
3. Entities Identified	4
4. Attributes and Data Dictionary	4
4.1 CUSTOMER	4
4.2 ADDRESS	5
4.3 CATEGORY	5
4.4 SUPPLIER	5
4.5 PRODUCT	6
4.6 ORDER	6
4.7 ORDER_LINE	6
4.8 PAYMENT	7
4.9 DELIVERY	7
5. Primary Keys	8
6. Foreign Keys	8
7. Relationships and Cardinality	9
7.1 Participation	9
7.2 How the 1:1 relationships are enforced	10
8. Entity Relationship Diagram	11
8.1 How to read the diagram	12
9. Design Decisions and Assumptions	12
10. Normalisation	13
11. Conclusion	14
References	14

1. Introduction

1.1 Purpose

This document presents the database design for WitleShop (Pty) Ltd, a fast-growing South African online retailer that sells electronics, clothing and home appliances through a web and mobile platform. The design identifies all entities, primary keys, foreign keys, relationships and cardinalities, and presents a complete Entity Relationship Diagram (ERD).

1.2 Scope

The database must manage six business areas: customers, products, orders, payments, deliveries and suppliers

1.3 Method

The design follows the standard conceptual-to-logical modelling process described by Coronel and Morris (2019): business requirements are analysed, entities and attributes are identified, relationships and cardinalities are defined, many-to-many relationships are resolved with associative entities, and the model is checked against the normal forms (Codd, 1970). The ERD uses Crow's Foot notation, a widely used variant of the entity relationship model introduced by Chen (1976).

2. Summary of Business Requirements

The key requirements extracted from the case study are summarised below.

Area	Requirement	Design response
Customers	Must register before ordering; unique e-mail; several delivery addresses; many orders	CUSTOMER (UNIQUE Email); separate ADDRESS entity; 1:M to ORDER
Products	One category and one supplier per product; categories and suppliers have many products	Foreign keys CategoryID and SupplierID in PRODUCT
Orders	One customer per order; many products per order; products appear in many orders	ORDER table and ORDER_LINE bridge entity
Payments	Exactly one payment per order; methods Card, EFT, PayFast	PAYMENT with UNIQUE OrderID; CHECK on PaymentMethod
Deliveries	Exactly one delivery per order, to one of the customer's registered addresses	DELIVERY with UNIQUE OrderID and a link to ADDRESS checked against the order's customer
Order status	Pending, Shipped, Delivered or Cancelled	CHECK constraint on OrderStatus

3. Entities Identified

Nine entities were identified. Seven come directly from the business areas in the case study; two (ADDRESS and ORDER_LINE) are required to satisfy normalisation and relationship rules.

#	Entity	Description	Type
1	CUSTOMER	Registered buyer	Strong entity
2	ADDRESS	Delivery address of a customer	Dependent on CUSTOMER through FK
3	CATEGORY	Product grouping	Strong entity
4	SUPPLIER	Company that supplies products	Strong entity
5	PRODUCT	Item for sale	Strong entity
6	ORDER	A purchase by one customer	Strong entity
7	ORDER_LINE	Products contained in each order	Associative (bridge) entity
8	PAYMENT	Payment for an order	Strong entity
9	DELIVERY	Delivery of an order	Strong entity

Why ADDRESS is a separate entity. A customer may have several delivery addresses. Storing them as columns or a list inside CUSTOMER would create a multi-valued attribute and violate First Normal Form (Elmasri and Navathe, 2016).

Why ORDER_LINE is needed. An order can contain many products and a product can appear in many orders. A many-to-many relationship cannot be stored directly in a relational database, so it is broken into two one-to-many relationships through a bridge table that holds the foreign keys of both parents (Coronel and Morris, 2019). ORDER_LINE also stores the attributes that belong to the relationship itself: Quantity, UnitPrice and LineTotal.

4. Attributes and Data Dictionary

The data dictionary below lists every attribute with its data type, key role, constraints and meaning. PK = primary key, FK = foreign key, UK = unique key.

4.1 CUSTOMER

A person who has registered on the WitleShop web or mobile platform. Registration is mandatory before an order can be placed.

Attribute	Data type	Key	Constraints	Description
CustomerID	INT	PK	Auto-generated	Unique customer identifier (surrogate key)
FullName	VARCHAR(100)		NOT NULL	Customer's full name
Email	VARCHAR(150)	UK	NOT NULL, UNIQUE	Login/contact e-mail; must be unique

Attribute	Data type	Key	Constraints	Description
PhoneNumber	VARCHAR(20)		NOT NULL	Contact number
DateOfRegistration	DATE		NOT NULL, default today	Date the customer registered

4.2 ADDRESS

A delivery address belonging to one customer. Created as a separate entity because a customer can store several addresses.

Attribute	Data type	Key	Constraints	Description
AddressID	INT	PK	Auto-generated	Unique address identifier
CustomerID	INT	FK	NOT NULL	Owner of the address (references CUSTOMER)
AddressLine1	VARCHAR(150)		NOT NULL	Street number and name
AddressLine2	VARCHAR(150)		NULL allowed	Complex, building or unit (optional)
Suburb	VARCHAR(80)		NULL allowed	Suburb
City	VARCHAR(80)		NOT NULL	City / town
Province	VARCHAR(40)		NOT NULL	South African province
PostalCode	VARCHAR(10)		NOT NULL	Postal code
Country	VARCHAR(60)		NOT NULL, default South Africa	Country

4.3 CATEGORY

A grouping of products, e.g. Electronics, Clothing and Home Appliances.

Attribute	Data type	Key	Constraints	Description
CategoryID	INT	PK	Auto-generated	Unique category identifier
CategoryName	VARCHAR(60)		NOT NULL, UNIQUE	Category name
Description	VARCHAR(255)		NULL allowed	Short description of the category

4.4 SUPPLIER

A company that supplies products to WitleShop.

Attribute	Data type	Key	Constraints	Description
SupplierID	INT	PK	Auto-generated	Unique supplier identifier
SupplierName	VARCHAR(100)		NOT NULL	Registered supplier name
ContactPerson	VARCHAR(100)		NULL allowed	Person to contact at the supplier

Attribute	Data type	Key	Constraints	Description
Email	VARCHAR(150)		NULL allowed	Supplier e-mail
PhoneNumber	VARCHAR(20)		NULL allowed	Supplier phone number

4.5 PRODUCT

An item that WitleShop sells. Each product has exactly one category and one supplier.

Attribute	Data type	Key	Constraints	Description
ProductID	INT	PK	Auto-generated	Unique product identifier
ProductName	VARCHAR(150)		NOT NULL	Product name
Description	TEXT		NULL allowed	Detailed description
Price	DECIMAL(10,2)		NOT NULL, ≥ 0	Current selling price (ZAR)
StockQuantity	INT		NOT NULL, ≥ 0	Units currently in stock
CategoryID	INT	FK	NOT NULL	References CATEGORY
SupplierID	INT	FK	NOT NULL	References SUPPLIER

4.6 ORDER

A purchase made by one registered customer.

Attribute	Data type	Key	Constraints	Description
OrderID	INT	PK	Auto-generated	Unique order identifier
CustomerID	INT	FK	NOT NULL	Customer who placed the order
OrderDate	DATE		NOT NULL, default today	Date the order was placed
OrderStatus	VARCHAR(15)		CHECK in (Pending, Shipped, Delivered, Cancelled)	Current state of the order
TotalAmount	DECIMAL(12,2)		NOT NULL, ≥ 0	Order total (ZAR)

4.7 ORDER_LINE

Associative entity that records which products are on which order. It resolves the many-to-many relationship between ORDER and PRODUCT: each row is one product line on one order.

Attribute	Data type	Key	Constraints	Description
OrderLineID	INT	PK	Auto-generated	Unique order line identifier
OrderID	INT	FK	NOT NULL	References ORDER
ProductID	INT	FK	NOT NULL	References PRODUCT; (OrderID, ProductID) is UNIQUE
Quantity	INT		NOT NULL, > 0	Number of units ordered
UnitPrice	DECIMAL(10,2)		NOT NULL, ≥ 0	Price per unit at the time of purchase
LineTotal	DECIMAL(12,2)		NOT NULL, ≥ 0	Quantity × UnitPrice (derived)

4.8 PAYMENT

The payment made for an order. Each order has exactly one payment record.

Attribute	Data type	Key	Constraints	Description
PaymentID	INT	PK	Auto-generated	Unique payment identifier
OrderID	INT	FK	NOT NULL, UNIQUE	Order being paid for (enforces 1:1)
PaymentDate	DATE		NULL until paid	Date payment was made
PaymentMethod	VARCHAR(10)		CHECK in (Card, EFT, PayFast)	How the customer paid
PaymentStatus	VARCHAR(10)		CHECK in (Pending, Completed, Failed, Refunded)	State of the payment (values assumed)
AmountPaid	DECIMAL(12,2)		NOT NULL, ≥ 0	Amount received (ZAR)

4.9 DELIVERY

The delivery of an order to one of the customer's registered addresses. Each order has exactly one delivery record.

Attribute	Data type	Key	Constraints	Description
DeliveryID	INT	PK	Auto-generated	Unique delivery identifier
OrderID	INT	FK	NOT NULL, UNIQUE	Order being delivered (enforces 1:1)
AddressID	INT	FK	NOT NULL	Destination; must be the customer's own address
DeliveryDate	DATE		NULL until scheduled	Delivery date
DeliveryStatus	VARCHAR(15)		CHECK in (Preparing, Dispatched, In Transit, Delivered, Failed)	State of the delivery (values assumed)
CourierName	VARCHAR(80)		NULL allowed	Courier company

Attribute	Data type	Key	Constraints	Description
TrackingNumber	VARCHAR(50)		NULL allowed	Courier tracking reference

5. Primary Keys

Every entity has a primary key that uniquely identifies each row and can never be NULL. Surrogate keys (system-generated integers) are used because they are compact, stable and never change, unlike natural values such as e-mail addresses (Silberschatz, Korth and Sudarshan, 2019).

Entity	Primary key	Notes
CUSTOMER	CustomerID	Surrogate key. Email is a candidate key, enforced with UNIQUE.
ADDRESS	AddressID	Surrogate key.
CATEGORY	CategoryID	Surrogate key. CategoryName is UNIQUE.
SUPPLIER	SupplierID	Surrogate key.
PRODUCT	ProductID	Surrogate key.
ORDER	OrderID	Surrogate key.
ORDER_LINE	OrderLineID	Surrogate key. (OrderID, ProductID) is a UNIQUE candidate key so a product appears only once per order; Quantity records how many.
PAYMENT	PaymentID	Surrogate key. OrderID is UNIQUE (1:1 with ORDER).
DELIVERY	DeliveryID	Surrogate key. OrderID is UNIQUE (1:1 with ORDER).

6. Foreign Keys

Foreign keys implement the relationships and enforce referential integrity: a foreign key value must match an existing primary key in the parent table (Codd, 1970). In this design every foreign key is mandatory (NOT NULL).

Foreign key	In table	References	Rule enforced
CustomerID	ADDRESS	CUSTOMER	Every address belongs to one customer
CustomerID	ORDER	CUSTOMER	Every order is placed by one registered customer
CategoryID	PRODUCT	CATEGORY	Each product belongs to exactly one category
SupplierID	PRODUCT	SUPPLIER	Each product is supplied by exactly one supplier
OrderID	ORDER_LINE	ORDER	Each order line belongs to an order
ProductID	ORDER_LINE	PRODUCT	Each order line refers to a product
OrderID	PAYMENT	ORDER (UNIQUE)	One payment per order

Foreign key	In table	References	Rule enforced
OrderID	DELIVERY	ORDER (UNIQUE)	One delivery per order
AddressID	DELIVERY	ADDRESS	Delivery goes to a registered address

7. Relationships and Cardinality

Cardinality describes how many instances of one entity can relate to instances of another (Coronel and Morris, 2019). The design contains five one-to-many relationships, two one-to-one relationships and one many-to-many relationship. The many-to-many relationship is implemented as two one-to-many relationships through ORDER_LINE (R5a and R5b).

#	Relationship	Type	Read as	Implemented by
R1	CUSTOMER has ADDRESS	1:M	One customer has many addresses; each address belongs to one customer	FK CustomerID in ADDRESS
R2	CUSTOMER places ORDER	1:M	One customer places many orders; each order belongs to one customer	FK CustomerID in ORDER
R3	CATEGORY classifies PRODUCT	1:M	One category has many products; each product has one category	FK CategoryID in PRODUCT
R4	SUPPLIER supplies PRODUCT	1:M	One supplier supplies many products; each product has one supplier	FK SupplierID in PRODUCT
R5	ORDER contains PRODUCT	M:N	An order has many products; a product appears in many orders	Bridge table ORDER_LINE
R5a	ORDER contains ORDER_LINE	1:M	One order has one or many lines; each line belongs to one order	FK OrderID in ORDER_LINE
R5b	PRODUCT included_in ORDER_LINE	1:M	One product can be on many lines; each line refers to one product	FK ProductID in ORDER_LINE
R6	ORDER has PAYMENT	1:1	One order has one payment; one payment belongs to one order	FK OrderID (UNIQUE) in PAYMENT
R7	ORDER delivered_by DELIVERY	1:1	One order has one delivery; one delivery belongs to one order	FK OrderID (UNIQUE) in DELIVERY
R8	ADDRESS used_for DELIVERY	1:M	One address can be used for many deliveries; each delivery goes to one address	FK AddressID in DELIVERY

7.1 Participation

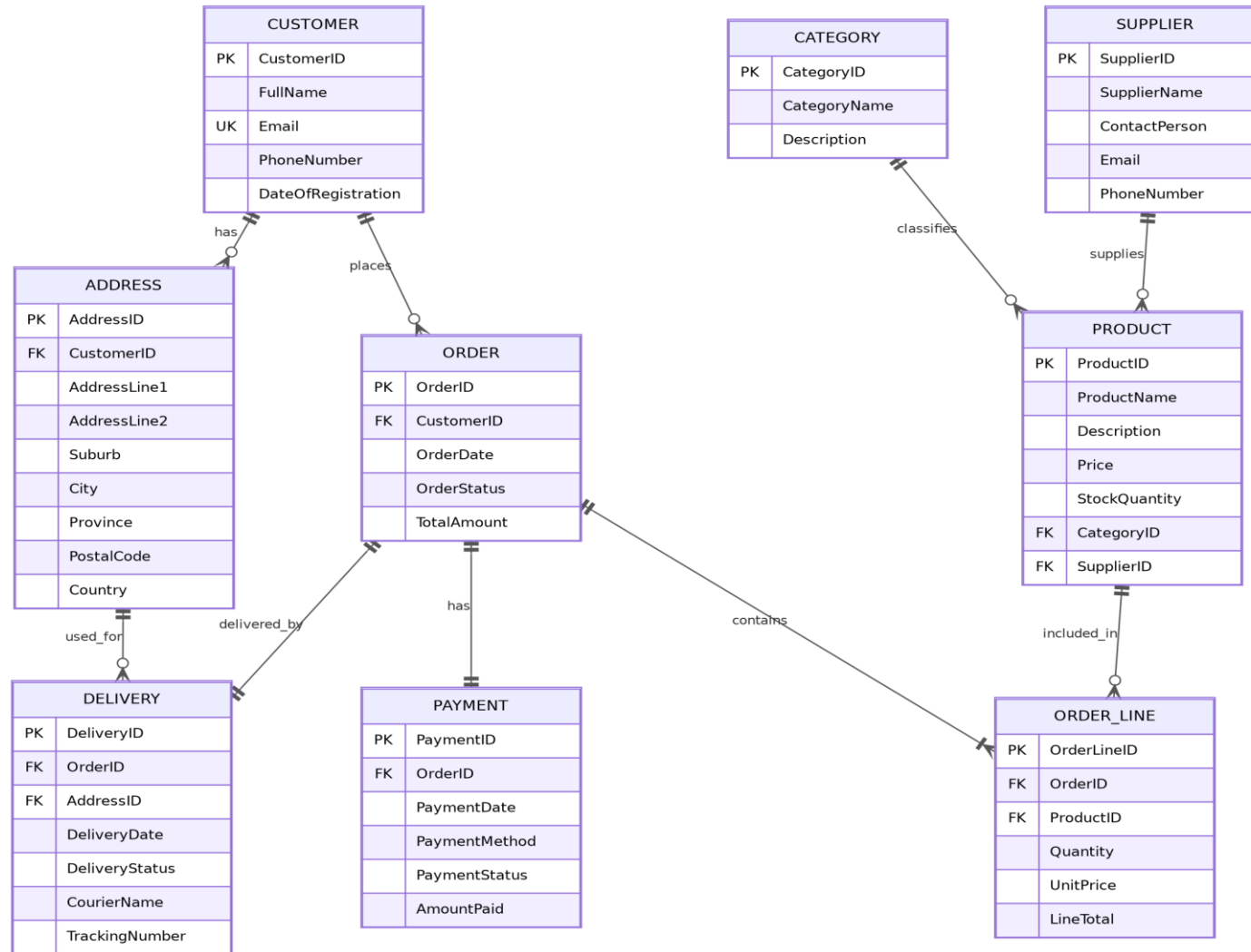
- **Mandatory on the “many” side:** every ADDRESS needs a customer, every ORDER needs a customer, every PRODUCT needs a category and supplier, and every PAYMENT and DELIVERY needs an order (all foreign keys are NOT NULL).
- **Optional on the “one” side:** a customer may exist without orders, a category or supplier may exist before any products are linked, and an address may exist without ever being used for delivery.

-
- **An order must contain products:** an order has one or more ORDER_LINE rows (shown as one-or-many on the ERD).

7.2 How the 1:1 relationships are enforced

A one-to-one relationship is created by placing a foreign key in one table and adding a UNIQUE constraint on it. Because OrderID is unique in PAYMENT (and in DELIVERY), the same order cannot appear twice, so an order can have at most one payment and one delivery. The NOT NULL constraint guarantees each of those records belongs to an order.

8. Entity Relationship Diagram



8.1 How to read the diagram

Symbol	Meaning
Two short bars	Exactly one (mandatory)
Circle and crow's foot O<	Zero or many (optional, many)
Bar and crow's foot <	One or many (at least one)
PK (left column)	Primary key attribute
FK (left column)	Foreign key attribute
UK (left column)	Unique key (no duplicate values allowed)
Verb on a line	Name of the relationship, e.g. "places", read from the "one" side towards the "many" side

Relationship names (the verbs on the lines) are standard practice in ERDs, because they make each relationship readable as a sentence, for example "one CUSTOMER places zero or many ORDERS" (Chen, 1976; Coronel and Morris, 2019). The brief does not require them, so they can be removed without changing the structure or the cardinalities of the model.

9. Design Decisions and Assumptions

- **Price history.** UnitPrice is stored in ORDER_LINE so that later changes to PRODUCT.Price do not alter past orders.
- **Derived amounts.** LineTotal (Quantity × UnitPrice) and TotalAmount (the sum of an order's LineTotals) can be calculated from other data. They are kept because the case study lists TotalAmount and because stored totals make reporting and auditing simpler. They should be calculated by the application or a database trigger so they cannot drift out of date. This is a deliberate, controlled denormalisation.
- **Delivery address ownership.** A simple foreign key from DELIVERY to ADDRESS would allow an order to be delivered to another customer's address. To enforce the rule "delivery must be linked to one of the customer's registered addresses", the SQL implementation uses a trigger that compares the address's CustomerID with the order's CustomerID (see Section 11).
- **Status values.** Order statuses and payment methods are taken from the case study. The case study does not list payment or delivery statuses, so the values Pending/Completed/Failed/Refunded and Preparing/Dispatched/In Transit/Delivered/Failed are assumed and may be changed.
- **Controlled lists.** CHECK constraints are used for the short fixed lists. If WitleShop expects those lists to change often, they could be moved to lookup tables.
- **Naming.** Entities are written in capitals (ORDER_LINE) and attributes in PascalCase (CustomerID). ORDER is a reserved SQL keyword, so the table is named Orders in the database script.

-
- **Personal information.** Customer data is personal information in terms of the Protection of Personal Information Act (Republic of South Africa, 2013). The design therefore stores only the data the business needs; access control and encryption should be handled at implementation.

10. Normalisation

Normalisation reduces redundancy and prevents update, insert and delete anomalies (Elmasri and Navathe, 2016). The design satisfies Third Normal Form (3NF), apart from the derived amounts described in Section 9.

Normal form	How the design satisfies it
1NF	All attributes are atomic and there are no repeating groups. Multiple addresses and multiple order lines are stored in their own tables.
2NF	Every non-key attribute depends on the whole primary key. All tables use a single-column surrogate key, so partial dependencies cannot occur.
3NF	No non-key attribute depends on another non-key attribute. Category and supplier details live in their own tables instead of being repeated in PRODUCT. The only exceptions are the derived amounts LineTotal and TotalAmount, which are kept deliberately.

11. Conclusion

The WitleShop database is modelled with nine entities in Third Normal Form. Every requirement in the case study is represented: customers can hold multiple addresses and orders; products belong to one category and one supplier; orders and products are linked through ORDER_LINE; and each order has exactly one payment and one delivery, the latter restricted to the customer's own addresses. The ERD, key definitions and cardinalities provide a sound basis for implementation and for later extension, such as returns, discounts or product reviews.

References

- Chen, P.P. (1976) 'The entity-relationship model: toward a unified view of data', *ACM Transactions on Database Systems*, 1(1), pp. 9–36.
- Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377–387.
- Coronel, C. and Morris, S. (2019) *Database systems: design, implementation, and management*. 13th edn. Boston: Cengage Learning.
- Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of database systems*. 7th edn. Harlow: Pearson.
- Republic of South Africa (2013) *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Printer.
- Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) *Database system concepts*. 7th edn. New York: McGraw-Hill Education.